1. The right mindset

This is an applied software-engineering subject. The exam rewards students who can THINK and WRITE code, not those who memorise definitions. Treat your revision the same way:

- Spend more time at the keyboard than with the slides. If you only read, you will recognise concepts but not produce them under pressure.
- Aim for understanding, not coverage. One topic learned deeply is worth ten topics skimmed.
- Explain things out loud. If you cannot teach it to a classmate in 60 seconds, you do not understand it yet.

2. Understand how marks are distributed

Knowing the shape of the paper helps you plan revision intelligently.

Cognitive level	What it expects from you	Share of marks
Remember	Recall a fact, term, or definition.	~20%
Understand	Explain in your own words, compare, summarise.	~45%
Apply	Read, write, or fix code. Choose the right syntax.	~35%

Two-thirds of the marks reward EXPLAINING and APPLYING. Pure memorisation will not get you a strong grade on its own.

2.1 Where the marks come from

The course covers the full stack — frontend, asynchronous JavaScript, Vue, PHP backend, MySQL, authentication, security, and mobile deployment. Plan revision in rough proportion to topic weight:

Priority	Topic area	Why it earns marks
High	Backend stack — REST, PHP/Slim, PDO/MySQL, JWT, security	Largest single block of structured marks.
High	Vue.js — components, reactivity, router, props	Large structured question; also a popular MCQ area.
Medium	Asynchronous JavaScript — Fetch, Promises, async/await	Strong combination of MCQ + code-writing marks.
Medium	JavaScript & DOM	Foundational; needed for every later topic.
Medium	Web architecture & HTML/CSS	Fast, easy marks if you have practised the basics.

3. What "mastery" looks like for each topic

For each chapter, ask yourself: can I do these things WITHOUT looking at notes? If not, that is where to spend your next study session.

3.1 Web fundamentals & UI

- Describe how a browser and a server exchange information for one page load.
- Name the role of frontend, backend, database, and API in a sentence each.
- Write a small HTML page that uses semantic elements correctly.
- Make a layout change responsively using a CSS media query.

3.2 JavaScript and the DOM

- Select elements by id, by CSS selector, and as a list.
- Attach a click handler that updates the page.
- Create a new element, set its text, and append it to a container.
- Read and write input field values.

3.3 Asynchronous JavaScript

- Define concept
- Use fetch to retrieve JSON and read it.
- Convert a .then() chain into an async/await function with try/catch.
- advantage and disadvantage

3.4 Vue.js

- Framework concept, components
- Single File Component
- Use ref, v-model, v-for, and props in a small component.

3.5 Backend, database & security

- concept of RESTful API .
- Write a simple Slim route (any HTTP verb) that reads input and returns JSON.
- Insert a row into MySQL using a PDO prepared statement.
- JWT authentication flows
- threats
- Explain at a high level how a web project becomes a mobile app (Capacitor).

Techniques to avoid

- Highlighting slides without writing anything yourself — it feels productive but teaches little.
- Re-watching every lecture from start to finish — slow and low-yield.
- All-night cramming — costs you more marks the next day than it could possibly add.
- Memorising entire code blocks without understanding them — one slight syntax twist and you lose all the marks.